# MS1 Report

Team name: Caml Slayers  Project: Slay the Caml

Members and NetIDs: Nicholas Lennon (ncl46), Carter Chamberlain (cjc423), Grace Jin (gsj33), Kiyam Merali (km942), Yuan Gao (yg626).

Repository: https://github.com/gracejinsotrue/cs3110-final-project  Communication: team text chat and Slack (#general, #brainstorming, #to-do).

The public repository README lists all five members, and its history shows a separate README contribution from each. The shared team contract is in the repository.

## Our team contract

### What we hope to get out of the project

We want to build a fun OCaml game while practicing collaborative design, testing, and integration. Each member needs a substantive component and timely help when work blocks someone else.

### Communication and meetings

We use text for short or urgent messages and Slack for longer discussions and code. We expect replies within 12 hours, except for messages sent at night. We meet weekly, virtually or in person, according to availability in our shared calendar. Anyone unable to attend gives at least 12 hours notice in the text group.

### Decisions

For major design choices, each teammate writes ideas before discussion. We listen respectfully, seek a workable compromise, and use a majority vote when the options conflict or cannot both be built. This process gives everyone a chance to be heard and lets work continue.

### Commitments and problems

We track tasks in the Slack #to-do canvas. A teammate who is stuck or expects to miss a commitment tells the text group. Missed work is reopened for another teammate; blocking work takes priority over nonblocking tasks. The original owner later takes on a task from the person who helped. If a problem persists after a recovery plan, we will ask the professor for team coaching.

### Review and integration

Contributors write explanatory commits and tests for their changes. We review code at weekly meetings. Documentation and weekly progress conversations keep backup owners familiar with each component. Interface changes are announced in Slack before dependent code is changed.

### AI use

AI may assist code development only when a human inspects, understands, and personally tests every generated line and its tests before committing. The contributor discloses AI-assisted files to the team, for example with a file comment. We will discuss disagreements at a team meeting and follow the course AI and academic-integrity rules.

### Revisiting the contract

We will review the agreement in depth at our third meeting and keep the repository copy open to agreed changes as issues arise.

## Our project proposal

Slay the Caml is a terminal-based, turn-based deck-building game inspired by the structure of Slay the Spire. The player begins a run with a small starter deck, limited health, and a visible route through fights, shops, rest stops, and a boss. At each map choice, the player weighs risk against possible rewards. The objective is to reach and defeat the boss before health runs out.

In combat, the screen shows the player's health, block, energy, numbered hand, and each enemy's next intent. The player chooses cards to attack, defend, draw, or apply effects such as Vulnerable. Cards cost energy, so playing one card can prevent another. Ending the turn causes the announced enemy move to occur; block and status effects change the damage taken. A win grants a choice among new cards, which changes the deck used in later fights. A loss ends the run. These decisions connect: map choices determine encounters, rewards shape the deck, and the deck changes the player's options in combat.

The finished game will use data files for cards and enemies so content can grow without changing combat code. A seeded random generator will make shuffles, encounters, and rewards reproducible for tests and players. A run state will track the current node, health, gold, and deck; saving and loading between encounters will let someone pause and return. A simple terminal interface keeps the focus on game rules and makes the complete path playable without a graphics toolkit. If core play is stable, the team can consider additional content or persistent unlocks; these are extensions, not prerequisites for a coherent game.

This scope combines several substantial pieces: an interpreter for card effects, enemy behavior and status timing, JSON parsing and validation, map and reward generation, run persistence, and a user interface that connects them. The work can be split into independently testable modules with small shared types and interfaces. That makes the project comparable in overall effort to A1 and A2 combined while allowing us to build a smaller playable version first.

## Our backup idea

If the main game proves infeasible early, we would build a Clarkson-themed course review dating simulator. A player answers CS 3110 questions during a sequence of conversations; correct answers increase affinity, incorrect answers decrease it, and different endings follow from the score. The questions and branching dialogue can be implemented with OCaml data types and a plain terminal interface, without relying on the deck-building engine or external card content.

## Initial responsibilities

Each primary owner builds, checks, and connects the named component. The backup reviews its interface and tests, follows progress at weekly meetings, and can help with a handoff.

| Area | Primary owner | Backup owner | What the owner will deliver |
|---|---|---|---|
| Combat engine | Nicholas Lennon | Yuan Gao | Card effect interpreter; energy, hand, draw and discard state; turn cycle; win/loss checks; combat tests and an interface for the UI. |
| Status and enemy AI | Yuan Gao | Nicholas Lennon | Vulnerable and later status rules; enemy intent patterns and tests; agreed damage and status timing with combat. |
| Content and loading | Carter Chamberlain | Grace Jin | Validated JSON definitions for cards and enemies; clear load errors; content tests; build, run, and content instructions in README. |
| Map and run progression | Kiyam Merali | Carter Chamberlain | Map choices, rewards and run state; later branching, shop, rest, and save/load; tests for legal choices and deterministic rewards. |
| UI and integration | Grace Jin | Kiyam Merali | Input parser, readable terminal screens, top-level game loop, integration of real modules, parser tests, and an end-to-end playthrough. |

Shared interfaces. The team will agree on cross-module data types and the first .mli signatures together before parallel implementation. Components receive their data and seeded random state explicitly. Grace's main loop connects the modules; renderers return strings that can be tested without a live game.

## Where we will start

First implementation for MS2. We will build one complete, playable terminal fight and card reward. The user starts a seeded run, chooses a fight on a tiny fixed map, sees a numbered hand and an enemy's announced move, plays cards or ends a turn, and receives clear errors for illegal commands without changing game state. Winning offers a card choice that changes the deck; losing ends the run without a reward. The first version will use one enemy, a starter deck, and a few cards. Branching maps, shops, bosses, and saves come after this path works.

Integration and success check. The team will first agree on small shared types and interfaces, then connect real JSON content to combat, status and enemy AI, the terminal loop, run state, and the reward screen. The slice is complete when a teammate who did not build the feature can play from start through win or loss without a crash, see the announced enemy move occur, and choose a reward only after winning. We will run module tests and one scripted end-to-end playthrough, and record actual results for MS2.

Early uncertainty and experiment. We need to confirm that our JSON card and enemy definitions can be loaded and validated consistently on every teammate's setup. In the first few days, each member will build a minimal Dune project with Yojson, ANSITerminal, and OUnit2; Carter will supply a small valid fixture and several malformed ones. We will check that the valid fixture loads, bad effect tags and costs give useful errors, and the same seed produces the same hand and reward. If a dependency or format causes trouble, the first fight can use a hardcoded fixture and plain text while we simplify the format or installation steps.

## Each member’s active MS2 contribution

**Nicholas Lennon**
Implements one-enemy combat, card play, turn order, and win/loss; tests damage through block, insufficient energy, and invalid hand selections.

**Yuan Gao**
Implements the enemy's repeating visible intent and Vulnerable timing; tests the intent cycle and damage adjustment with Nicholas.

**Carter Chamberlain**
Creates the small JSON card and enemy set, loader, validation checks, and run instructions; tests valid and malformed content.

**Kiyam Merali**
Implements the fixed map, run transitions, and deterministic reward choice; tests legal choices, deck updates, and no reward after a loss.

**Grace Jin**
Implements command parsing and terminal screens, connects the modules in the main loop, and leads a full playthrough and UI checks.